DRL Sessions 2 & 3 — Multi-Armed Bandits (Exam Handout)

Expected Learnings: MAB formulation · Action-value methods · ε-greedy · Incremental updates · Non-stationarity · Optimistic IV · UCB · Bandit Gradient · Contextual Bandits

1 · The k-Armed Bandit Problem

Formal Definition

At each time step t you choose one of k actions (arms). You receive a numerical reward drawn from the action's **stationary** probability distribution. **Objective:** maximise *expected total reward* over some time horizon.

Three defining features

1. **Unknown rewards** — the true value $q^*(a) = \mathbb{E}[R_t \mid A_t = a]$ is not known upfront.
2. **Repeated choice** — you face the same problem over and over and can learn from experience.
3. **Observable outcome** — after choosing a you observe the reward R_t.

Why we study MAB: It is the simplest setting where the *exploration–exploitation trade-off* appears. Every concept (value estimates, greedy selection, ε-greedy, UCB) reappears in full MDPs.

2 · Action-Value Methods (Sample Average)

Estimated value of action a at time t

$$Q_t(a) \;=\; \frac{\text{sum of rewards when } a \text{ was selected before } t}{\text{number of times } a \text{ was selected before } t} \;=\; \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}$$

By the Law of Large Numbers, $Q_t(a) \to q^*(a)$ as the count $\to \infty$.

Greedy Action Selection

$$A_t^* \;=\; \arg\max_a Q_t(a)$$

Always exploiting — never discovers potentially better actions. Optimal only if initial estimates are already correct.

3 · Exploration vs. Exploitation — ε-Greedy

ε-Greedy Policy (Algorithm)

```
epsilon = 0.1            # small exploration rate
def get_action():
    if random() > epsilon:        # prob (1 - epsilon)
        return argmax Q(a)        # EXPLOIT: greedy
    else:                         # prob epsilon
        return random.choice(A)   # EXPLORE: uniform random
```

Probabilities for a specific action a (greedy action):

$$P(\text{greedy action selected}) = \underbrace{(1-\varepsilon)}_{\text{exploit branch}} + \underbrace{\frac{\varepsilon}{k}}_{\text{explore branch picks greedy by chance}}$$

$$P(\text{non-greedy action } a_i \text{ selected}) = \frac{\varepsilon}{k}$$

Worked Example 1 — from slides (must know)

Q: In ε-greedy with $k = 2$ actions and $\varepsilon = 0.5$, what is the probability that the greedy action is selected?

Method (law of total probability):

$$\begin{aligned} P(\text{greedy}) &= P(\text{greedy} \mid \text{greedy branch})\, P(\text{greedy branch}) \\ &\quad + P(\text{greedy} \mid \text{explore branch})\, P(\text{explore branch}) \\ &= 1 \times (1 - 0.5) + \tfrac{1}{2} \times 0.5 \\ &= 0.5 + 0.25 \;=\; \mathbf{0.75} \end{aligned}$$

General formula check: $(1 - \varepsilon) + \varepsilon/k = 0.5 + 0.5/2 = 0.75$✓

Effect of ε on performance

ε	Behaviour	Best when
0 (pure greedy)	Never explores; gets stuck at a sub-optimal arm	Rewards deterministic; initial estimates already good
Small (0.01–0.1)	Mostly exploits; finds optimum slowly but surely	Stationary, noisy rewards
Large (0.5)	Explores heavily; slow to exploit best arm	High variance rewards or non-stationary setting

10-armed testbed conclusion: $\varepsilon = 0.1$ finds the best arm faster early on; $\varepsilon = 0.01$ eventually performs better as its estimates become more accurate.

4 · Incremental Implementation (Session 3)

Incremental update rule

Given Q_n (estimate after $n - 1$ rewards) and the n-th reward R_n:

$$Q_{n+1} \;=\; Q_n + \frac{1}{n}\big[R_n - Q_n\big]$$

General form (step size α):

$$Q_{n+1} \;=\; Q_n + \alpha\big[R_n - Q_n\big]$$

$[R_n - Q_n]$ is the **error** (target − estimate). Multiply by step size; add to old estimate. No need to store all past rewards.

Constant vs. Varying Step Size

- $\alpha = 1/n$ (sample average): gives equal weight to all past rewards. Good for **stationary** problems.
- α = constant: gives more weight to *recent* rewards ($\alpha(1-\alpha)^{n-k}$ weight on reward k steps ago). Good for **non-stationary** problems.

5 · Non-Stationarity

Problem: most real RL environments change over time. ε-greedy with sample average ($\alpha = 1/n$) converges to the true value of the *original* distribution — not the current one.

Exponential recency-weighted average (α constant)

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i$$

Recent rewards get exponentially more weight. This is the standard approach for non-stationary bandits.

6 · Optimistic Initial Values

Idea: Set $Q_1(a) = +5$ (much higher than any real reward, e.g. max real reward ≈ 1). Every action is initially over-estimated.

Effect: Whichever action is tried first yields a real reward < 5. The agent is "disappointed" and tries other actions. All actions are explored early.

Caution (exam point):

- Works well for **stationary** problems only.
- In **non-stationary** settings it fails: once the initial exploration burst is over, the agent stops exploring, even if action values have since changed.

7 · Upper Confidence Bound (UCB)

UCB Action Selection

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Each term:

- $Q_t(a)$ — current value estimate of action a
- $N_t(a)$ — how many times a has been selected so far
- $c\sqrt{\ln t / N_t(a)}$ — **uncertainty bonus**: large when a is rarely tried; shrinks as a is tried more
- c — confidence level, controls how optimistic we are about uncertain actions

Intuition: UCB explores *intelligently* — it prefers actions that are *either high-valued or rarely tried*, not random ones. As t grows, $\ln t / N_t(a) \to 0$ for frequently tried arms — they are selected less often over time.

Limitation: UCB is harder to generalise beyond the bandit setting to full MDPs (unlike ε-greedy).

8 · Bandit Gradient Algorithm (High Level)

Instead of estimating action *values*, learn action *preferences* $H_t(a)$ directly.

Action selection — Softmax:

$$\pi_t(a) \;=\; \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}}$$

Preference update after selecting A_t and receiving R_t:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t))$$

$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) \quad \forall\, a \neq A_t$$

where $\bar{R}_t$ is the running average reward (the baseline).

Key idea: If $R_t > \bar{R}_t$ (better than average), increase preference for A_t; decrease for all others. Proofs excluded from exam scope.

9 · Contextual Bandits (Associative Tasks)

What are Contextual Bandits?

In the standard bandit, the same k arms exist in every step. In a **contextual bandit**, each step has a **context** (situation/state) s, and the best action can differ by context.
Example: A news recommender. Context = user profile + time of day. Best article to recommend changes with the context.
Policy: a mapping from context → best action (unlike standard MAB which maps nothing → action).
Why special approaches? If context switches are rapid, optimistic initial values and simple averaging fail. Need methods that generalise across contexts (e.g. linear regression on features, neural network policies).
Relation to full RL: Contextual bandits are the bridge from MAB to MDP. In an MDP, your action changes the *next* context — that dependency is absent in contextual bandits.

10 · Exam Worked Examples

Ex-2 — Which steps definitely/possibly used the ε case?

Setup: $k = 4$, ε-greedy, sample-average, $Q_1(a) = 0$ for all a.
Sequence: $A_1 = 1, R_1 = 1;\ A_2 = 2, R_2 = 1;\ A_3 = 2, R_3 = 2;\ A_4 = 2, R_4 = 2;\ A_5 = 3, R_5 = 0$

Compute Q before each step, then check if the action was greedy:

Step	$Q(1)$	$Q(2)$	$Q(3)$	$Q(4)$	Greedy?	ε case?
$t = 1$	0	0	0	0	Tie (any)	**Possibly** (tie → any arm is "greedy")
$t = 2$	1.0	0	0	0	$A^* = 1$	**Definitely** (chose 2, not greedy arm 1)
$t = 3$	1.0	1.0	0	0	Tie (1 or 2)	**Possibly** (arm 2 is tied for greedy)
$t = 4$	1.0	1.5	0	0	$A^* = 2$	Possibly (arm 2 is greedy; choosing it is fine)
$t = 5$	1.0	1.67	0	0	$A^* = 2$	**Definitely** (chose 3, not greedy arm 2)

Answer: ε case *definitely* occurred at $t = 2$ and $t = 5$. It *possibly* occurred at $t = 1$ (tie), $t = 3$ (tie), $t = 4$ (arm 2 was greedy so no ε needed).

Summary — All 4 Methods at a Glance

Method	Key idea	Best for
Greedy	Always pick highest $Q_t(a)$	Only if estimates are already good
ε-Greedy	Exploit $(1-\varepsilon)$, random explore ε	General-purpose; easy to tune
Optimistic IV	Start estimates high; forces exploration early	Stationary problems only
UCB	Exploit + uncertainty bonus	Stationary; smarter exploration than random
Gradient/Softmax	Learn preferences, not values	Policy-based; when value estimation is hard